The Journal of Open Source Software

HiGP: A high-performance Python package for Gaussian Processes

Hua Huang [1*], **Tianshi Xu** [2*], **Yuanzhe Xi** [2], and **Edmond Chow** [1¶]

1 School of Computational Science, Engineering, Georgia Institute of Technology, USA **2** Department of Mathematics, Emory University, USA ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: James Gaboardi

Reviewers:

- @Edenhofer
- @davidogara
- @Naeemkh

Submitted: 04 July 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Gaussian Processes (GPs) (Rasmussen & Williams, 2005) are flexible, nonparametric Bayesian models widely used for regression and classification tasks due to their ability to capture complex data patterns and provide uncertainty quantification. Traditional GP implementations often face challenges in scalability as data sizes increase. To address these challenges, HiGP, a high-performance Python package, is designed for efficient Gaussian Process regression and classification across datasets of varying sizes. HiGP uses iterative solvers to access large GP computations. It implements matrix-vector (MatVec) and matrix-matrix (MatMul) multiplication algorithms tailored to kernel matrices by exploiting their hierarchical low-rank structure (Cai et al., 2023; Huang et al., 2020; Xing & Chow, 2020). To improve the convergence of iterative methods, HiGP also integrates the Adaptive Factorized Nyström (AFN) preconditioner (Zhao et al., 2024) which is designed to be robust across a wide range of kernel hyperparameter values used during hyperparameter optimization. HiGP can be used with PyTorch and other Python packages, allowing it to be incorporated into common machine learning and data analysis workflows.

Gaussian Processes

For training points $\mathbf{X} \in \mathbb{R}^{n\times d}$, a noisy training observation set $\mathbf{y} \in \mathbb{R}^n$, and testing points $\mathbf{X}_* \in \mathbb{R}^{m\times d}$, a standard GP model assumes that the noise-free testing observations $\mathbf{y}_* \in \mathbb{R}^m$ follow a joint Gaussian distribution that depends on a set of parameters, including scale f, noise level s, and kernel parameters l. The GP model finds the optimal parameters $\Theta := (s, f, l)$ by minimizing the negative log marginal likelihood:

$$L(\Theta) = \frac{1}{2}\left(\mathbf{y}^\top \widehat{\mathbf{K}}^{-1}\mathbf{y} + \log|\widehat{\mathbf{K}}| + n\log 2\pi\right),$$

where $\widehat{\mathbf{K}}$ denotes the regularized kernel matrix. An optimization process usually requires the gradient of $L(\Theta)$:

$$\frac{\partial L}{\partial \theta} = \frac{1}{2}\left(-\mathbf{y}^\top \widehat{\mathbf{K}}^{-1}\frac{\partial \widehat{\mathbf{K}}}{\partial \theta}\widehat{\mathbf{K}}^{-1}\mathbf{y} + \mathrm{tr}\left(\widehat{\mathbf{K}}^{-1}\frac{\partial \widehat{\mathbf{K}}}{\partial \theta}\right)\right), \quad \theta \in \Theta.$$

Using preconditioned iterative methods with preconditioner $\mathbf{M} \approx \widehat{\mathbf{K}}$ is a common option Chen et al. (2023). In this approach, $\widehat{\mathbf{K}}^{-1}\mathbf{y}$ is approximated via the preconditioned conjugate gradient (PCG) method (Saad, 2003). To handle the logarithmic determinant and trace terms, they are first rewritten as

$$\log|\widehat{\mathbf{K}}| = \log|\mathbf{M}| + \log|\mathbf{M}^{-1/2}\widehat{\mathbf{K}}\mathbf{M}^{-1/2}|, \tag{1}$$

$$\mathrm{tr}(\widehat{\mathbf{K}}^{-1}\frac{\partial\widehat{\mathbf{K}}}{\partial\theta}) = \mathrm{tr}\left(\mathbf{M}^{-1}\frac{\partial\mathbf{M}}{\partial\theta}\right) + \mathrm{tr}\left(\widehat{\mathbf{K}}^{-1}\frac{\partial\widehat{\mathbf{K}}}{\partial\theta} - \mathbf{M}^{-1}\frac{\partial\mathbf{M}}{\partial\theta}\right). \tag{2}$$

The second component of each new expression is then estimated using the stochastic Lanczos quadrature (Ubaru et al., 2017) and the Hutchinson estimator (Hutchinson, 1989, p. Meyer:2021), respectively. The AFN preconditioner is used in HiGP to speed up the optimization process.

Statement of Need

Multiple GP packages have been released in recent years to utilize modern hardware and handle larger datasets, including GPyTorch (Gardner et al., 2018), GPflow van der Wilk et al. (2020), GPy (GPy, since 2012), and other packages. The motivation for HiGP's development is to utilize new numerical algorithms and parallel computing techniques to reduce computational complexity and to improve the computation efficiency of iterative methods in GP model training. Compared to existing packages, HiGP has three main advantages and contributions.

Firstly, HiGP addresses the efficiency of MatVec, the most performance-critical operation in iterative methods. For large 2D or 3D datasets (e.g. spatial data), the dense kernel matrix is compressed into a $\mathcal{H}^2$ matrix Hackbusch & Börm (2002) in HiGP, resulting in $\mathcal{O}(n)$ storage and computation costs. For large high-dimensional datasets, HiGP computes small blocks of the kernel matrix on-the-fly and immediately uses these blocks in MatVec before discarding them instead of storing them in memory. This on-the-fly mode allows HiGP to handle extremely large datasets on a computer with moderate memory size.

Secondly, HiGP adopts a scalable computational approach: iterative solvers with a robust preconditioner. HiGP adopts the newly proposed AFN preconditioner, which is designed for robust preconditioning of kernel matrices. Numerical experiments demonstrate that AFN can significantly improve the accuracy and robustness of iterative solvers for kernel matrix systems.

Lastly, HiGP uses accurate and efficient hand-coded gradient calculations. GPyTorch relies on the automatic differentiation (autodiff) provided in PyTorch to calculate gradients (Equation (2)). Although autodiff is convenient and flexible, it is very inefficient when used to evaluate derivatives for iterative methods, due to long chain rule expressions resulting from the iterations.

Design and Implementation

We implemented HiGP in Python 3 and C++ with the goal of providing both a set of ready-to-use out-of-the-box Python interfaces for regular users and a set of reusable high-performance multithreading computational primitives for advanced users.

The HiGP C++ code implements all performance-critical operations, including populating the matrix block $\mathbf{K}(X, Y; l)$ and $\partial\mathbf{K}(X, Y; l)/\partial l$, computing $\tilde{\mathbf{K}} = f^2\mathbf{K}(X, Y; l) + s\mathbf{I}$, and matrix products $\tilde{\mathbf{K}} \times B$ and $(\partial\tilde{\mathbf{K}}/\partial\theta) \times B$ in dense or $\mathcal{H}^2$ matrix form, preconditioned conjugate gradient (PCG) solver, and computations of loss and gradients in GP regression and GP classification.

HiGP Python code wraps the C++ units into four basic Python modules: `krnlmatmodule` for computing kernel matrices and its derivatives, `precondmodule` for PCG solver with AFN preconditioner, `gprproblemmodule` and `gpcproblemmodule` for computing the the loss and gradient for GP regression and classification. Advanced users can utilize `krnlmatmodule` and `precondmodule` modules to develop new algorithms for kernel matrices. The two modules `gprproblemmodule` and `gpcproblemmodule` allow a user to train a GP model with any gradient-based optimizer.

DRAFT

We further implemented two high-level modules `GPRModel` and `GPCModel` to simplify the training and use of GP models. These modules register the hyperparameters used in GP regression/classification as PyTorch parameters and set the gradients of PyTorch parameters in each step model the PyTorch optimizer. Listing 1 shows an example of defining and training a GP regression and using the trained model for prediction in just eight lines of code.

```
# Listing 1: HiGP example code of training and using a GPR model
gprproblem = higp.gprproblem.setup(data=train_x, label=train_y,
                                   kernel_type=higp.GaussianKernel)
model = higp.GPRModel(gprproblem)
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)
for i in ranges(max_steps):
    loss = model.calc_loss_grad()
    optimizer.step()
params = model.get_params()
pred = higp.gpr_prediction(train_x, train_y, test_x,
                           higp.GaussianKernel, params)
```

We note that the HiGP Python interfaces are *stateless*. This design aims to simplify the interface and decouple different operations. A user can train and use different GP models with the same or different data and configurations in the same file.

Aune, E., Simpson, D. P., & Eidsvik, J. (2014). Parameter estimation in high dimensional Gaussian distributions. *Statistics and Computing*, *24*(2), 247–263. https://doi.org/10.1007/s11222-012-9368-y

Cai, D., Huang, H., Chow, E., & Xi, Y. (2023). Data-driven construction of hierarchical matrices with nested bases. *SIAM Journal on Scientific Computing*, *46*(2), S24–S50. https://doi.org/10.1137/22m1500848

Chen, L., Chen, T., Detha, U., & Andersen, M. S. (2023). Towards scalable kernel-based regularized system identification. *2023 62nd IEEE Conference on Decision and Control (CDC)*, 1498–1504. https://doi.org/10.1109/CDC49753.2023.10384051

Gardner, J., Pleiss, G., Weinberger, K. Q., Bindel, D., & Wilson, A. G. (2018). GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. *Advances in Neural Information Processing Systems*, *31*.

GPy. (since 2012). *GPy: A Gaussian process framework in python*. http://github.com/SheffieldML/GPy.

Hackbusch, W., & Börm, S. (2002). Data-sparse approximation by adaptive $\mathcal{H}^2$-matrices. *Computing*, *69*(1), 1–35.

Hackbusch, W., Khoromskij, B. N., & Sauter, S. A. (2000). On $\mathcal{H}^2$-matrices. In *Lectures on applied mathematics: Proceedings of the symposium organized by the sonderforschungsbereich 438 on the occasion of karl-heinz hoffmann's 60th birthday, munich, june 30 - july 1, 1999* (pp. 9–29). Max Planck Institute for Mathematics in the Sciences; Springer.

Hensman, J., Fusi, N., & Lawrence, N. D. (2013). *Gaussian processes for big data*. 282–290. https://dl.acm.org/doi/10.5555/3023638.3023667

Huang, H., Xing, X., & Chow, E. (2020). H2Pack: High-performance h^2 matrix package for kernel matrices using the proxy point method. *ACM Transactions on Mathematical Software*, *47*(1), 1–29.

Hutchinson, M. F. (1989). A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, *18*(3), 1059–1076. https://doi.org/10.1080/03610918908812806

Matthews, A. G. de G., van der Wilk, M., Nickson, T., Fujii, Keisuke., Boukouvalas, A., León-Villagrá, P., Ghahramani, Z., & Hensman, J. (2017). GPflow: A Gaussian process

DRAFT

library using TensorFlow. *Journal of Machine Learning Research*, *18*(40), 1–6.

Pleiss, G., Gardner, J., Weinberger, K., & Wilson, A. G. (2018). Constant-time predictive distributions for Gaussian processes. *Proceedings of the 35th International Conference on Machine Learning*, *80*, 4114–4123.

Rasmussen, C. E., & Williams, C. K. I. (2005). *Gaussian processes for machine learning*. The MIT Press. https://doi.org/10.7551/mitpress/3206.001.0001

Saad, Y. (2003). *Iterative methods for sparse linear systems*. Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9780898718003

Ubaru, S., Chen, J., & Saad, Y. (2017). Fast estimation of $tr(f(A))$ via stochastic lanczos quadrature. *SIAM Journal on Matrix Analysis and Applications*, *38*(4), 1075–1099. https://doi.org/10.1137/16M1104974

van der Wilk, M., Dutordoir, V., John, S., Artemev, A., Adam, V., & Hensman, J. (2020). *A framework for interdomain and multioutput Gaussian processes*. https://arxiv.org/abs/2003.01115

Wenger, J., Pleiss, G., Hennig, P., Cunningham, J., & Gardner, J. (2022). Preconditioning for scalable Gaussian process hyperparameter optimization. *Proceedings of the 39th International Conference on Machine Learning*, 23751–23780.

Wilson, A. G., Dann, C., & Nickisch, H. (2015). *Thoughts on massively scalable Gaussian processes*.

Xing, X., & Chow, E. (2020). Interpolative decomposition via proxy points for kernel matrices. *SIAM Journal on Matrix Analysis and Applications*, *41*, 221–243. https://doi.org/10.1137/19m1258700

Zhang, M., Chen, T., & Mu, B. (2024). Asymptotic properties of generalized maximum likelihood hyper-parameter estimator for regularized system identification. *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 4961–4966. https://doi.org/10.1109/CDC56724.2024.10885816

Zhao, S., Xu, T., Huang, H., Chow, E., & Xi, Y. (2024). An adaptive factorized Nyström preconditioner for regularized kernel matrices. *SIAM Journal on Scientific Computing*, *46*(4), A2351–A2376. https://doi.org/10.1137/23M1565139

DRAFT